

Implemente em assembly Cortex-M uma função que verifica se determinado número inteiro é primo. A verificação consiste em calcular o resto da divisão por números primos já listados; se o número sendo testado for divisível por algum dos primos listados então este número **não** é primo. A ideia é que o usuário desta função vá testando os inteiros sequencialmente e montando uma listagem de primos em um vetor, a exemplo deste segmento de código em C:

```
uint32_t primes[100] = {2}; //primes[0] = 2
uint32_t n_primes = 1;
for (uint2_t i =3; i < 100; ++i) {
    if (check_prime(i,primes,n_primes) {
        primes[n_primes++] = i;
    }
}
```

A função que você irá desenvolver tem a seguinte declaração, e a implementação deve seguir rigorosamente a AAPCS:

```
bool check_prime(uint32_t iut, uint32_t * primes, uint32_t np);
```

Descrição dos parâmetros e retorno:

iut – integer-under-test número inteiro positivo a ser testado
primes – endereço inicial do vetor com os números primos já encontrados;
observar que todos os primos menores que *iut* estão presentes neste vetor.
np – quantos elementos há no vetor *primes* (≥ 1)

retorno: 1 se o número for primo e
 0 se não for primo (i.e. se for divisível por algum dos valores em *primes*)

Observe que a operação módulo (operador % em C) pode ser implementada da seguinte forma: se Dividendo (*Dv*) for dividido por Divisor (*Dr*) gerando Quociente (*Q*) e Resto (*R*) então:

$$Dv = Q \times Dr + R$$

portanto o Resto (resultado da operação módulo) pode ser calculado a partir de uma divisão, multiplicação e subtração:

$$R = Dv - (Q \times Dr)$$

Exemplo de uso:

Se o conteúdo do vetor *v* for {2, 3, 5}, a chamada:

```
check_prime(6, v, 3)
```

retorna 0.